CONTENTS

- Summary
- Requirements and how they are met
- Architecture
- The daily cycle
- When proxycheck.io is asked
- How the answer is resolved
- The tables
- Vendors and mappings
- proxycheck.io in detail
- The API
- Production safety
- AWS resources
- Operations and runbooks
- State on 15 Sep 2026
- What was done, in order
- Limitations and open items
- Files in the project
- Glossary

COMPLETE REFERENCE · AS BUILT AND VERIFIED 15 SEP 2026

IP Intelligence System Guide

Everything about the company IP source: what it is for, how every part works, why each decision was made, the evidence that it works, what was done along the way, and what is still open.

AWS account **221490242148** · **us-west-2** Database **ipintel_curated** Status **live**

Summary

The system gives every IP address the company sees a location and a proxy/VPN/bot verdict, keeps both current every day, and lets anyone look up any address, one or a thousand at a time.

- **Sources of addresses:** the production tables `raw.raw_tune_clicks` (ad clicks) and `unified.unified_session_summary` (website sessions). They are only ever read.
- **Answers from:** IP2Location DB11 and IP2Proxy PX12 (downloaded databases), FraudLogix decisions carried on tune clicks, ipinfo.io and ipwho.is (free APIs), and proxycheck.io (paid, $9.99 a month, 80,000 queries a day).
- **Kept in:** nine Iceberg tables in `ipintel_curated`, the main one being `ip_resolved`, one row per address with the answer to use.

- **Kept current by:** a daily load at 07:30 UTC for new addresses, a proxycheck.io run straight after (new-row addresses first, then a 70,000-address backfill), a 22:00 UTC top-up that uses the rest of the 80,000, and weekly maintenance.
- **Used through:** a Lovable app, an HTTPS API (`GET /lookup/{ip}` , `POST /lookup` for up to 1,000, table and control endpoints), and Athena.
- **Safety:** every project role sits inside a permissions boundary that makes writing to any production table, bucket or workgroup impossible; every run proves it and reviews its own AWS activity.

Requirements and how they are met

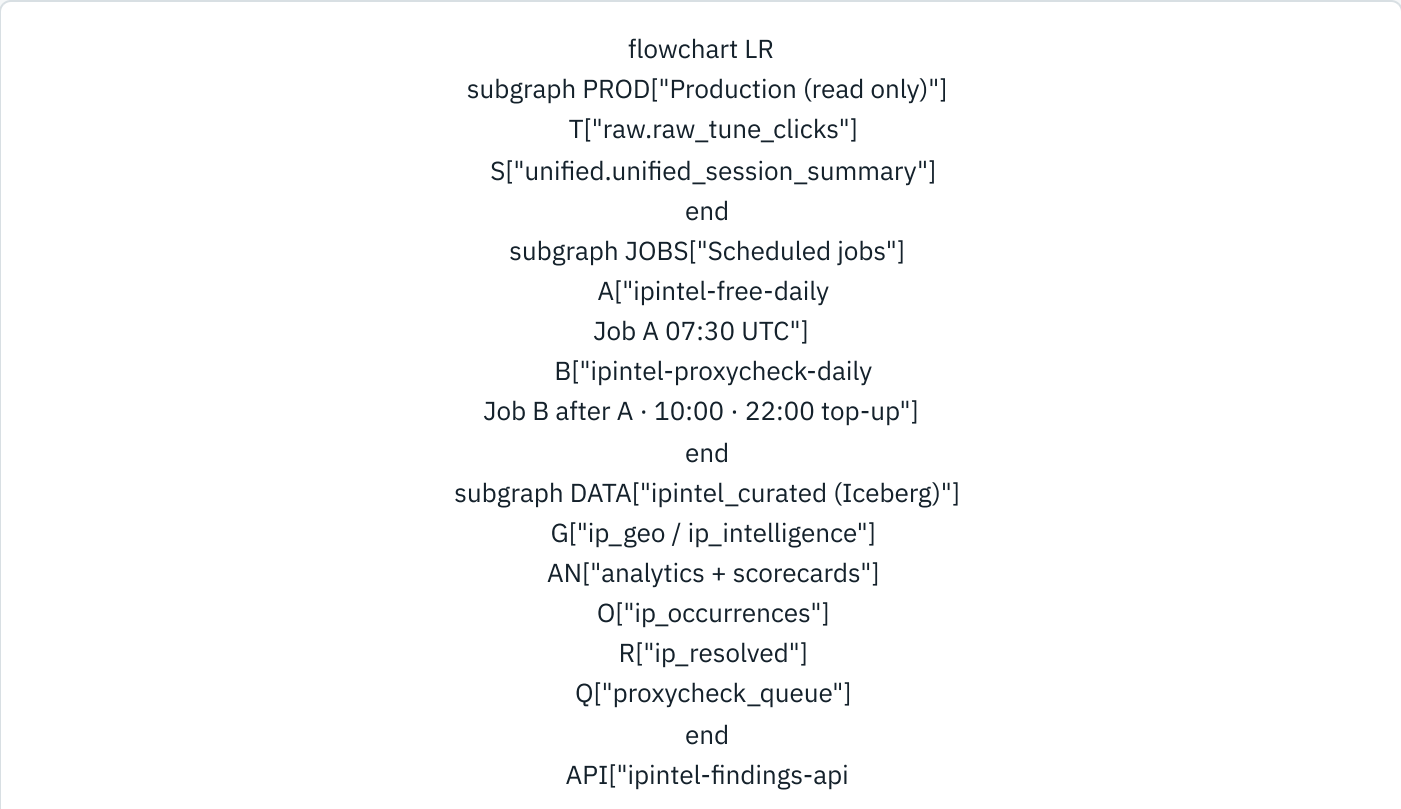
Each requirement the owner set, in the owner's terms, with where it lives. Evidence for every row is in `IP_PROJECT_VERIFICATION.md` .

REQUIREMENT	HOW IT IS MET
Do not touch `raw.raw_tune_clicks` or `unified.unified_session_summary` ; nothing in production modified, added or deleted	Read-only access enforced by AWS (permissions boundary); IAM simulator check before every run; CloudTrail review after every run
New rows in those tables drive the process	Job A reads each base table at a pinned version, detects change, finds new addresses and addresses whose rows changed
Look up our IP tables first; call proxycheck.io if the address is missing or the information is bad	One rule, shared by the API and the pipeline (below); the pipeline queues addresses and sends them first the same day
Daily job: new IPs to each vendor, compare, update vendor overview and unified tables	Job A: IP2Location, IP2Proxy, FraudLogix, ipinfo.io, ipwho.is; analytics, scorecards, `ip_resolved`
Unique IPs; existing appended, new inserted	MERGE on (address, vendor): inserted, added, updated; duplicate check every run
Separate daily job: 50,000 addresses with no proxycheck.io decision; bulk; self-loop; no data loss, no downtime, valid calls, no key exhaustion, no timeouts	Job B: 1,000 per request, responses saved before use, shared reservation counter, hand-over to a fresh copy, Iceberg snapshots so readers never see half a write. Budget later raised to 70,000 plus top-up
Use the full 80,000 queries a day	70,000 backfill + queue + lookups + 22:00 top-up keeping 2,000

REQUIREMENT	HOW IT IS MET
Base tables controllable from the UI	`/input-tables` endpoints and the Base Tables page, with validation
API usable directly, for one or many IPs, with geo and proxy	`GET /lookup/{ip}` and `POST /lookup`
Where does the IP occur	`ip_occurrences`, returned by every lookup
Correct information every time, without modifying vendor data	Vendor rows are kept as answered; `ip_resolved` and every lookup apply the same resolution rules
Nothing dummy or benchmark	Benchmark-only vendors and the old sample endpoints removed; every row comes from a vendor the system runs
Most safety options	Boundary, emergency stop, automatic pause, alarms and email alerts, one copy of each job, dead-letter queue, scan caps, audit log

Architecture

Three Lambda functions do all the work; everything they write lives in one Glue database and one S3 folder that belong to the project.





COMPONENT	WHAT IT DOES
`ipintel-free-daily`	Job A: new addresses, free vendors, analytics, scorecards, occurrences, resolved answers, queue. Also validates base tables and runs weekly maintenance. Python 3.12, 15 min, 1 GB, one copy at a time
`ipintel-proxycheck-daily`	Job B: queue, backfill and top-up batches to proxycheck.io; folds saved live answers into the tables. Same limits
`ipintel-findings-api`	The HTTPS API behind the app and other systems. 60 s, 512 MB, bearer token
`ipintel-pipeline-state` (DynamoDB)	Settings and run history: safety state, job settings, base tables, the daily query counter, run logs, audit log
`ipintel-sources` (DynamoDB)	The vendor registry shown on Vendor Sources
S3 `…/ip-intelligence/`	Iceberg table files, proxycheck.io plans and responses, saved live answers, staging, exports, archives
Athena workgroups	`ipintel_etl` for jobs (10 GB per query cap), `ipintel` for the API (20 GB cap)
EventBridge Scheduler	Four schedules (operations)

COMPONENT	WHAT IT DOES
SNS `ipintel-alerts` + CloudWatch	Email alerts and six alarms

The daily cycle

Job A — the incremental load (07:30 UTC)

1. Stops if the pipeline is paused or the job is switched off (a manual run ignores the switch).
2. Removes staging tables older than 3 hours left by any crashed run.
3. Asks the IAM simulator whether its role could write to production. Any "yes" fails the run and pauses the pipeline.
4. Takes the enabled, validated base tables and records the latest Iceberg snapshot of each. If none changed since the last successful run, it stops ("skipped").
5. **New addresses:** every valid IPv4 in any base table, at the pinned snapshot, that IP2Location has not answered yet. A full comparison is used, so rows that arrive late or are rewritten are never missed.
6. **FraudLogix targets:** new addresses plus addresses with clicks in the last 3 days, because a FraudLogix decision can change.
7. Looks up IP2Location DB11 and IP2Proxy PX12 (range joins on the downloaded databases) and FraudLogix (latest decision per address from tune clicks).
8. Asks ipinfo.io and ipwho.is about addresses new since 14 Sep that they have not answered, up to 500 each per run, paced.
9. Counts what will change (inserted / added / updated / unchanged), then MERGEs into `ip_geo` and `ip_intelligence`. `created_at` is never overwritten.
10. Refreshes `ip_intelligence_analytics` and `ip_geo_analytics` for the affected addresses.
11. Recomputes `ip_occurrences` from every base table and merges it (first seen only moves earlier, last seen only later).
12. Recalculates both vendor scorecards over addresses asked of every running vendor (at least 1,000) and exports them for the app.
13. Recomputes `ip_resolved` for every address and merges only rows that changed.
14. **Queue:** every address new or touched since the last successful run that needs proxycheck.io goes into `proxycheck_queue` with its reason.

15. Runs 12 checks (below) and a CloudTrail review. If any fails, progress is not recorded, so the next run repeats the same work.
16. Records each base table's new snapshot as its watermark and starts Job B.

Job A checks: pipeline cannot write to production · no address twice for a vendor · no table shrank · every new address is in `ip_geo` · IP2Location has every new address · IP2Proxy has every new address · updated geo rows kept `created_at` · updated intelligence rows kept `created_at` · every address has exactly one resolved answer · every new address has an occurrence row · every address needing proxycheck.io is queued · nothing outside the project was touched in 24 hours.

Job B — proxycheck.io (after Job A; 10:00 fallback; 22:00 top-up)

1. Same pause, switch and permission checks. Reads the key from Secrets Manager and proxycheck.io's own usage page (plan, limit, queries used).
2. **Plans the day once:** picks up to 70,000 × 99% addresses without a proxycheck.io answer, in this order: queued from base tables, first seen today, no conclusion and seen in the last 90 days, no conclusion and older, then clean. Private and reserved addresses are never picked. Saves the list as batch files of 1,000 and a manifest.
3. **Queue batches:** on every run, queued addresses not in today's batches become extra batches sent first, so addresses queued after the plan still go out the same day. They may use half the lookup reserve.
4. **Top-up batches (22:00 run only):** the next addresses in the same order, sized to what is left of today's limit minus 2,000, judged by the larger of our counter and proxycheck.io's.
5. For each batch without a saved response: reserve its queries atomically, send 1,000 addresses in one request, save the response file immediately, then account for addresses charged twice.
6. With under 3 minutes left, or under 10 minutes when the merge is still to do, it starts a fresh copy of itself and stops (at most 30 hand-overs a day; a failed hand-over lands in the dead-letter queue with an alarm).
7. Merges every saved response not yet applied, plus every answer saved by API lookups in the last 7 days, into `ip_intelligence` and `ip_geo`; refreshes analytics, scorecards, exports and `ip_resolved`. Writes an "applied" marker per file only if every check passes.
8. Closes queue rows whose answer is now in the tables (or that proxycheck.io had no record for), then runs its checks and the CloudTrail review.

Job B checks: pipeline cannot write to production · no duplicates · no table shrank · `created_at` kept (geo and intelligence) · one resolved answer per address · every queued base-table address was sent · nothing outside the project touched.

Why it cannot lose data or overspend

FAILURE	WHAT HAPPENS
Crash after a response is saved	The next copy skips that batch and merges the file
Crash between answer and save	That batch is sent again; up to 1,000 queries wasted, no answer lost
Two copies at once	Impossible: each function allows one running copy
Budget reached	Stops; tomorrow's plan continues from the same order
Key rejected	Stops, marks the key invalid, emails
A check fails	Nothing is marked done; the next run repeats the work
Readers during a write	Iceberg serves the last complete version until the new one commits

When proxycheck.io is asked

One rule, written twice in the same order: `lookup.needs_proxycheck` in the API and `pcqueue.reason_sql` in the pipeline.

1. Not a public address (0/8, 10/8, 100.64/10, 127/8, 169.254/16, 172.16/12, 192.0.0/24, 192.0.2/24, 192.168/16, 198.18/15, 198.51.100/24, 203.0.113/24, 224/3) → never.
2. Refresh requested (API only) → yes, `refresh`.
3. proxycheck.io has already answered → no.
4. Not in the IP tables → yes, `new_address`.
5. No location → yes, `no_location`.
6. Proxy status `no_answer`, `vendors_disagree` or `flagged_unconfirmed` → yes, that status.
7. Location vendors disagree → yes, `location_disagree`.
8. Otherwise → no.

Tested 15 Sep: 9 of 9 offline cases; through the live API 4 of 4 should-call cases called, 3 of 3 must-not cases not called, 2 of 2 repeats not charged again; bulk endpoint the same.

How the answer is resolved

Vendor rows are never changed to make them agree. The answer to use is worked out from them by fixed rules, identically in `resolved.py` (SQL, for `ip_resolved`) and `lookup.py`

(Python, for lookups). A comparison of 1,000 addresses found 0 differences.

Location

- A vendor "answered" if it gave a state, a city or coordinates. GeoJS never votes (it repeats MaxMind); it is also no longer running.
- Answers are grouped by lower-cased state and the first word of the city.
- The group with the most vendors wins; ties go to the group holding the best location-scorecard score, then to the group whose first vendor id sorts first.
- The best-scoring vendor in the winning group supplies every field.
- Status: `agreed` if the group has 2+ vendors; `single_source` if only one vendor votes; otherwise `vendors_disagree`.

Proxy

- A vendor "answered" if `is_proxy` is not empty. It is "flagged" if any of proxy, VPN, Tor, datacenter, bot, compromised, scraper is true.
- If proxycheck.io answered, its flags are used: `confirmed` when another answering vendor has the same flagged/clean verdict, `proxycheck_only` when no other vendor answered, `proxycheck_overrides` when all others disagree.
- Without proxycheck.io: `clean_by_vendors` when every answering vendor says clean (all flags false); `vendors_disagree` when some flag and some do not; `confirmed_by_published_range` when all flag and a published range agrees (no range data is loaded now); otherwise `flagged_unconfirmed`. No answers at all is `no_answer`.
- ISP and ASN come from proxycheck.io when present, otherwise the first answering vendor by id.

The tables

All in `ipintel_curated`, Iceberg format, partitioned by `bucket(ip_address, 64)` (the queue by 16), stored under `s3://datalake-landingzone-221490242148-us-west-2/ip-intelligence/iceberg/`.

TABLE	ONE ROW PER	ROWS 15 SEP	WRITT
`ip_resolved`	address	4,956,914	A, B

TABLE	ONE ROW PER	ROWS 15 SEP	WRITT
ip_geo	address + vendor	5,039,539	A, B
ip_intelligence	address + vendor	9,567,985	A, B
ip_intelligence_analytics	address + vendor	= intelligence	A, B
ip_geo_analytics	address + vendor	106,938 before the top-up; grows daily	A, B
ip_occurrences	address + base table	6,543,790	A
proxycheck_queue	address	grows daily	A que closes
vendor_overview_geo	vendor	4	A, B
vendor_overview_proxy	vendor	3	A, B

TABLE	ONE ROW PER	ROWS 15 SEP	WRITT

Supporting tables in the same database: `db11_bucketed` and `px12_bucketed` (the downloaded IP2Location and IP2Proxy databases, bucketed by /16 prefix for fast range joins), `qualified_lead_ips` (customer addresses and confirmed humans, used as truth for location and false positives). Older tables from earlier phases (`ip_pool` , `pc_live` , `ip_dim` and similar) are not used by the running system.

Conventions: every yes/no is three-state (true, false, empty = no answer). Every timestamp is UTC `YYYY-MM-DDTHH:MM:SSZ` ; FraudLogix times were converted from Pacific time on 15 Sep.

S3 folder layout

```
ip-intelligence/
  iceberg/table/                          table data and metadata
  proxycheck/plan/dt=date/                 manifest.json, batch-NNNNN (backfill), batch-q…
  proxycheck/responses/dt=date/            raw proxycheck.io response per batch, saved bef
  proxycheck/applied/dt=date/              marker once a batch is merged and checked
  proxycheck/live/dt=date/                 answers from API lookups (single and bulk), fol
  proxycheck/live/by-ip/ip.json            latest API answer per address, so a repeat look
  proxycheck/live-applied/                    markers for folded live answers
  staging/run/                             short-lived working tables, removed after each
  serving/ipintel_curated/                    scorecard exports the app reads
  archive/                                    backups: removed benchmark vendors, FraudLog
  athena-results/                             query output
```

The frozen copies `unified.ip_geo` , `unified.ip_intelligence` , `unified.ip_geo_analytics` , `unified.ip_intelligence_analytics` , `unified.vendor_overview_geo` and `unified.vendor_overview_proxy` were built on 11 Sep and never changed since. Nothing reads them now.

Vendors and mappings

VENDOR	KIND	GIVES	COVERAGE
IP2Location DB11	downloaded database	location	every address

VENDOR	KIND	GIVES	COVERAGE
IP2Proxy PX12	downloaded database	proxy (only listed proxies answer)	every address asked; only listed proxies answer (0.3% of all addresses)
FraudLogix	decisions on tune clicks	proxy, VPN, Tor, datacenter, bot	tune-click addresses only
ipinfo.io, ipwho.is	free APIs	location	addresses new since 14 Sep, up to 500 each a day
proxycheck.io	paid API, 80,000/day	location and proxy, risk score, ISP, ASN	queue, backfill, top-up and lookups

FraudLogix mapping

Latest decision per address. Not `FLAGGED` → every flag false, reason `ACCEPTED`. `FLAGGED` with reason `PROXY` or `MOBILE_PROXY_SIGNALS` → proxy; `VPN` → VPN; `TOR` or `TOR_EXIT_NODE` → Tor; `DATA_CENTER` → datacenter; `BOT_GENERATED` or `NON_HUMAN_SCRIPT` → bot; `MASKED_DEVICES` → no flag, `is_masked_device` true (a device fingerprint signal, not a bot signal). Clicks with no decision → no answer.

proxycheck.io mapping

`proxy: yes` → proxy; `vpn: yes` → VPN; type Tor → Tor; type **Hosting** → datacenter; type Compromised Server → compromised; type Scraper → scraper; operator anonymity, proxy or VPN → anonymous; bot is always empty. **Business is not datacenter:** in a test of 400 real addresses, 112 were Business, mostly Charter, Cox, AT&T, CenturyLink and Verizon Business home and office broadband. `last seen unix` becomes `updated_at`.

Removed vendors

MaxMind GeoLite2, DB-IP, GeoJS and the published cloud ranges ran only in the 8–10 Sep benchmark on 2,105 addresses. Removed on 14 Sep with the owner's approval and switched off in the registry; every removed row is in `archive/benchmark-vendors-2026-09-14/`. They can be reintroduced as real monthly loads, which would need downloads (and a free MaxMind licence key).

proxycheck.io in detail

ITEM	VALUE
Plan	$9.99 a month, tier "Subscription", 80,000 queries a day, 5 burst tokens

ITEM	VALUE
Key	Secrets Manager `ipintel/proxycheck-paid` (the free key `ipintel/proxycheck` was used for testing). Never shown or accepted by the app
Request	`POST https://proxycheck.io/v2/?key=…&vpn=3&asn=1&risk=1&seen=1&days=90` , body `ips=a,b,c` , up to 1,000 addresses
Why those flags	`vpn=3` is the only setting that returns proxy and VPN separately; `seen=1` returns last-seen time; `asn=1` and `risk=1` return network and risk
Cost quirk	An address that is both proxy and VPN is charged twice (504 queries measured for 500 addresses); 1% is reserved extra and topped up afterwards
Firewall quirk	Python's default User-Agent is refused; every request sends its own
Status "warning"	Still carries full answers; treated as success
Usage page	`/dashboard/export/usage/` gives plan, limit and queries today; it lags 1–14 minutes, so our own counter is the one that decides
Counter	DynamoDB item `quota / date` : `reserved` (everything), `batch_reserved` (jobs), `live_reserved` (API). Each reservation is a conditional update that fails if it would pass the cap
Split	Backfill 70,000 (`daily_budget`); lookup reserve 10,000 (`live_reserve`); queue may use budget + 5,000; top-up at 22:00 fills to 80,000 − 2,000 (`topup_keep`)
Throughput	1,000 addresses in about 1–2 s; the 50-batch backfill and its merge fit in one Lambda run on 15 Sep; the 28-batch top-up ran separately

The API

One Lambda function behind a Function URL, authenticated by a bearer token (Lambda environment variable `API_TOKEN` ; the app holds it as `IPINTEL_API_TOKEN`).

ENDPOINT	PURPOSE
`GET /health`	Up and which mode
`GET /lookup/{ip}` · `?refresh=true`	One address, with proxycheck.io when the rule says so
`POST /lookup`	Up to 1,000 addresses; `proxycheck` , `refresh` , `detail` options

ENDPOINT	PURPOSE
`GET /tables/{name}`	ip-resolved, ip-geo, ip-intelligence, ip-geo-analytics, ip-intelligence-analytics, ip-occurrences, proxycheck-queue, vendor-overview-geo, vendor-overview-proxy
`/safety` , `/safety/pause` , `/safety/resume`	Emergency stop and safety state
`/input-tables` (+ `/{id}` , `/{id}/validate`)	Base tables: list, add, change, remove, validate
`/catalog/databases|tables|columns`	Pickers for adding a base table
`/jobs` , `/jobs/{id}` , `/jobs/{id}/run` , `/jobs/{id}/runs`	Job settings, run now, history
`/quota`	Today's queries, limit, key status
`/sources` , `/sources/{id}` , `/diagnose`	Vendor registry

Rules the server enforces: the budget cap (at most 70,000), a reason for pause and resume, no resume while a check fails, no removing the last enabled base table, the pipeline's own database cannot be a base table, no run while paused, the key can never be set through the API, mode has no endpoint. Every change through the API is written to the audit log for 400 days.

Mode. The safety item's `mode` is `live` (owner-approved 15 Sep): table pages read `ipintel_curated` . Setting it to `shadow` points the four per-vendor table pages back at the frozen `unified` copy. Lookups always read `ipintel_curated` .

Production safety

The pipeline cannot harm production because AWS will not let it, and every run proves that before it starts.

The fence

Every project role has the permissions boundary `ipintel-boundary` . Whatever a role's own policy says, the boundary explicitly denies:

- any Glue write outside `ipintel_curated` ;
- any S3 write outside `datalake-landingzone-221490242148-us-west-2/ip-intelligence/*` , and every bucket-settings change;
- queries in any Athena workgroup other than `ipintel` and `ipintel_etl` ;

- any Lambda outside `ipintel-*`;
- loosening the boundary itself, and deleting the project's own safety resources.

ROLE	USED BY	PROOF
`ipintel-builder-role`	Every build step (profile `ipintel-builder`)	IAM simulator 61 / 61 refusals
`ipintel-pipeline-role`	Both jobs	IAM simulator 64 / 64 refusals; re-checked at the start of every run
`ipintel-api-role`	The API	Hardened: its old write access to every `unified` table was removed
`ipintel-scheduler-role`	Schedules	Can only start the two job functions

Every run

- Before: the IAM simulator is asked whether the pipeline role could update or delete the production tables, write their buckets, change bucket notifications or lifecycle, or query in `primary`. Any "allowed" fails the run and pauses everything.
- After: CloudTrail is read for every call the job functions made in 24 hours; any write aimed outside the project is a failed check and an automatic pause.

Found during the audit

A review of the shared login's history found that this project ran only 3 read queries in the `primary` workgroup. Two production changes in that period came from other users of the shared login (the console and a separate assistant session): an `UPDATE` of `unified.lu_connector` and a `DROP` of `de_temp.klaviyo_uct_events`. They were reported to the owner and are unrelated to this project. Also noted: the `BounceEvents` EventBridge rule matches every S3 write in the account but has no target today.

AWS resources

All created for this project and tagged `project=ipintel` unless noted.

TYPE	NAME
IAM	`ipintel-boundary` (policy); roles `ipintel-builder-role`, `ipintel-pipeline-role`, `ipintel-scheduler-role`; existing `ipintel-api-role` hardened
Lambda	`ipintel-free-daily`, `ipintel-proxycheck-daily`, `ipintel-findings-api`

TYPE	NAME
Scheduler	`ipintel-free-daily` 07:30 · `ipintel-proxycheck-daily` 10:00 · `ipintel-proxycheck-topup` 22:00 · `ipintel-weekly-maintenance` Sun 03:00 (all UTC)
DynamoDB	`ipintel-pipeline-state` , `ipintel-sources`
SQS / SNS	`ipintel-proxycheck-dlq` / `ipintel-alerts` (one confirmed email subscription)
CloudWatch alarms	`ipintel-free-daily-run-failed` , `ipintel-proxycheck-daily-run-failed` , `ipintel-free-daily-no-run-26h` , `ipintel-proxycheck-dlq-not-empty` , `ipintel-proxycheck-quota-90pct` , `ipintel-maintenance-run-failed`
Athena	workgroups `ipintel` (20 GB cap), `ipintel_etl` (10 GB cap)
Glue	database `ipintel_curated`
S3	folder `ip-intelligence/` in `datalake-landingzone-221490242148-us-west-2` (versioning on; Intelligent-Tiering after 30 days; nothing auto-deleted)
Secrets Manager	`ipintel/proxycheck-paid` , `ipintel/proxycheck`

DynamoDB items in `ipintel-pipeline-state` : `safety/state` (paused, mode, checks, last audit), `job/free-daily` and `job/proxycheck-daily` (enabled, schedule, budget, key secret, top-up settings), `base_table/*` (table, columns, validation, watermark), `quota/`*`date`* (counter), `run#`*`job`*`/`*`time`* (every run's counts and checks), `audit/`*`time`* (every UI change).

Cost. proxycheck.io $9.99 a month. AWS usage is small: the jobs read a few GB in Athena a day (Athena charges $5 per TB, so cents a day) *(projection)*; Lambda, DynamoDB, SQS, SNS and S3 are within cents to a few dollars a month *(projection)*.

Operations and runbooks

WHEN (UTC)	WHAT
07:30 daily	Job A, then Job B
10:00 daily	Job B fallback (does nothing if already done)
22:00 daily	Job B top-up
Sun 03:00	Maintenance: `OPTIMIZE … REWRITE DATA USING BIN_PACK` and `VACUUM` on the 8 main tables (table names must be unquoted)

Runbooks

A job failed

Read the run's message and checks on Daily Jobs (or `GET /jobs/{id}/runs`). Nothing half-done is recorded as done. Fix the cause, then Run now. Answers already saved to S3 are merged on the next run.

A safety check failed and the pipeline paused itself

Do not resume. Compare the role and boundary policies with the copies in `build/iam/`; find what granted access; restore; run the job by hand to re-check; then resume with a reason.

The key was rejected

Renew the plan or put the new key in `ipintel/proxycheck-paid` (Secrets Manager, us-west-2). Run the proxycheck job by hand; the budget page should show "Key working".

Something in the dead-letter queue

A hand-over did not start. Run the proxycheck job by hand; it continues from the saved batches. Then clear the message from the queue.

Switch the table pages back to the frozen copy

Set `mode` to `shadow` on the `safety/state` item. The API picks it up within a minute. Nothing else changes.

Undo a data correction

Removed benchmark rows: `archive/benchmark-vendors-2026-09-14/` (JSON). FraudLogix timestamps before conversion: `archive/fraudlogix-timestamps-before-utc-2026-09-15/` (Parquet). Recent table versions can also be read with Iceberg time travel for 5 days (maintenance removes older ones).

Add a vendor back as a real source

Load its database into `ipintel_curated` under the table name in `vendors.OPTIONAL_*_DATABASES`, switch it on in the registry, and the daily job starts asking it about new addresses and scoring it.

Deploying code

Pipeline: zip `build/pipeline/src/*.py` (common, vendors, apply, resolved, pcqueue, job_free, job_proxycheck) and update both job functions. API: zip `build/api/src/*.py` (app, athena_tables, bulk, control, lookup) and update `ipintel-findings-api`. Always with the `ipintel-builder` profile, never the default profile.

State on 15 Sep 2026

MEASURE	VALUE
Addresses (from 4,956,909 in the base tables, plus live lookups)	4,956,914
Missing any expected vendor, duplicates, rows without analytics	0
proxycheck.io answers	77,710
Clean by every vendor	4,030,801
No conclusion yet (of which seen in 90 days)	848,403 (44,775)
Flagged	5,688
Location agreed by 2+ vendors	33,744
Queries used today	about 78,000 of 80,000
Days until every no-conclusion address is asked, at about 78,000 a day *(projection)*	about 11

Findings from the review report (*IP Intelligence Table Review*, 15 Sep): proxycheck.io judged 95% of the FraudLogix flags in the first backfill clean; most location disagreements are the same state and a nearby city; the busiest VPN-flagged addresses are Cloudflare privacy-relay exits; FraudLogix flags 6 of 210 confirmed humans as bots (2.9%).

What was done, in order

8–10 Sep 2026

Benchmark of location and proxy vendors on real addresses: MaxMind GeoLite2, DB-IP, GeoJS, IP2Location, IP2Proxy, ipinfo.io, ipwho.is, proxycheck.io (free key) and published cloud ranges. Scoring settled on "Agreement", not "Accuracy", with customer addresses as truth for location.

11 Sep

All 4.96 million addresses loaded into six `unified.*` tables from IP2Location, IP2Proxy and FraudLogix; Athena-backed table API and a first live lookup built. Found and fixed: proxycheck "warning" treated as failure, cursor tokens breaking in URLs, INSERT not allowed on bucketed tables.

11–14 Sep

Plans revised with the owner into `IP_PROJECT_FINAL_PLAN.md`: new tables in `ipintel_curated`, `unified` untouched, two daily jobs, UI control. proxycheck.io paid plan bought; key stored as `ipintel/proxycheck-paid`. Found: proxycheck "Business" type is home broadband, not datacenter.

14 Sep

Audit of the shared login's workgroup history. Permissions boundary and builder, pipeline and scheduler roles created and proven. Six Iceberg tables built. Job A and Job B built; first runs failed on a SQL detail (a WITH clause inside UNION ALL) before any write, then passed. Control API, schedules, alarms, paid key switch. Added `ip_occurrences` and `ip_resolved`; weekly maintenance; benchmark-only vendors removed with approval; staging sweep; private addresses excluded.

15 Sep 07:30

First scheduled runs: 108 new addresses, all 11 checks; first 50,000-address backfill in one run, all checks.

15 Sep morning

Table pages switched to the live tables (owner approval). Old benchmark endpoints removed. Review report on all tables. FraudLogix timestamps converted from Pacific to UTC; `false_positive_rate_pct` set to the approved confirmed-human definition. One proxycheck.io rule for API and base-table inserts, with `proxycheck_queue`.

15 Sep late morning

Full 80,000 a day: backfill 70,000 and a 22:00 top-up (first top-up sent 28 batches; its final CloudTrail review hit a rate limit after the merge, and the review is now paced). Bulk `POST /lookup` built and tested (1,000 addresses, 0 differences from `ip_resolved`). Alert email subscribed and confirmed.

Limitations and open items

ITEM	EFFECT	WHAT WOULD CHANGE IT
Open Base-table queue path not yet seen end to end on a real new row	Its first real use is the next 07:30 run; both jobs' checks fail loudly if an address is missed	Read the Daily Jobs cards after that run
Open Lovable prompt not yet applied	The app still has pages built on removed endpoints	Paste `LOVABLE_PROMPT_FINAL_APP.md`

ITEM	EFFECT	WHAT WOULD CHANGE IT
Production side `unified_session_summary` not rewritten since 13 Sep	No new website-session addresses	That table's owner
Tables update daily	An address can take up to about a day to appear	API lookup, or Run now
Bot flags from one vendor	No second opinion on bots	A vendor with bot classification, or a bot-labelled pool for precision and recall
Most locations from IP2Location alone	City approximate	The backfill adds proxycheck.io locations; reloading GeoLite2 or DB-IP as real sources
Proxy scorecard sample skewed toward FraudLogix flags, and its score rewards not flagging	Use it to compare vendors, not as accuracy	The sample widens as the backfill reaches clean addresses
proxycheck.io answers are not re-checked	An answer can age	A re-check policy once every address has one (about 11 days)
proxycheck.io's daily reset time is not published	Our counter resets at midnight UTC	The counter also respects proxycheck.io's own usage, so it cannot overspend
90% alert most days	Expected email	Switch that alarm off if unwanted
API lookups after 22:00 have about 2,000 queries	Large bulk lookups late in the day may be answered from tables only	Raise `topup_keep`
Monthly database reloads (IP2Location, IP2Proxy) not automated	Databases age month by month	Needs download approval; a monthly load job

Files in the project

PATH	WHAT IT IS
`docs/ip-intelligence-user-manual.html`	How to use everything
`docs/ip-intelligence-system-guide.html`	This guide
`IP_PROJECT_FINAL_PLAN.md`	The agreed plan

PATH	WHAT IT IS
`IP_PROJECT_VERIFICATION.md`	Requirement-by-requirement proof
`ATHENA_REVIEW_QUERIES_LIVE.md`	Ready-made queries and column meanings
`LOVABLE_PROMPT_FINAL_APP.md`	The one prompt for the app
`build/pipeline/src/`	`common.py` (AWS helpers, safety, run log), `vendors.py` (vendor lookups and mappings), `apply.py` (merges, analytics, scorecards, checks), `resolved.py`, `pcqueue.py`, `job_free.py`, `job_proxycheck.py`
`build/pipeline/sql/`	Table definitions (10–19) and `checks/` (every verification query)
`build/pipeline/*.py`	`run_sql.py`, `watch_run.py`, `compare_resolved.py`, `test_base_table_trigger.py`, `remove_benchmark_vendors.py`
`build/api/src/`	`app.py` (routes), `lookup.py` (single lookup, rules), `bulk.py`, `athena_tables.py`, `control.py`
`build/iam/`	Boundary and role policies, simulator proofs, backups of earlier versions

Glossary

`Iceberg`	A table format in S3 that allows safe updates (MERGE), versions (snapshots) and time travel. Readers always see a complete version.
`MERGE`	One SQL statement that updates matching rows and inserts new ones: the "append if it exists, insert if it doesn't" of the requirements.
`snapshot / watermark`	A table version. Job A records the snapshot it read from each base table, so the next run knows what changed.
`staging table`	A short-lived working table for one run, removed afterwards.
`permissions boundary`	An AWS ceiling on what a role can ever do, whatever its own policy grants.
`IAM simulator`	An AWS service that answers "would this role be allowed to do this?" without doing it.

`CloudTrail`	AWS's log of every API call made in the account.
`hand-over`	A Lambda run starting a fresh copy of itself to continue, before its 15-minute limit.
`dead-letter queue`	Where a failed asynchronous start is kept so it is not lost.
`reservation counter`	The DynamoDB item every proxycheck.io request reserves from first, so two callers can never both take the last queries.
`resolved answer`	The one answer to use per address, from fixed rules over every vendor's answer.
`no conclusion`	No vendor answered, vendors disagree, or a flag has nothing confirming it.
`agreement`	How often a vendor matches the others; the scorecards measure agreement, not accuracy.
`confirmed human`	An address a qualified customer lead used (`qualified_lead_ips.confirmed_human`); a bot flag on it is a false positive.
`mode`	`live` : table pages read `ipintel_curated` ; `shadow` : they read the frozen `unified` copy.

Kept in the project folder as `docs/ip-intelligence-system-guide.html` . Companion: IP Intelligence User Manual. Evidence: `IP_PROJECT_VERIFICATION.md` .